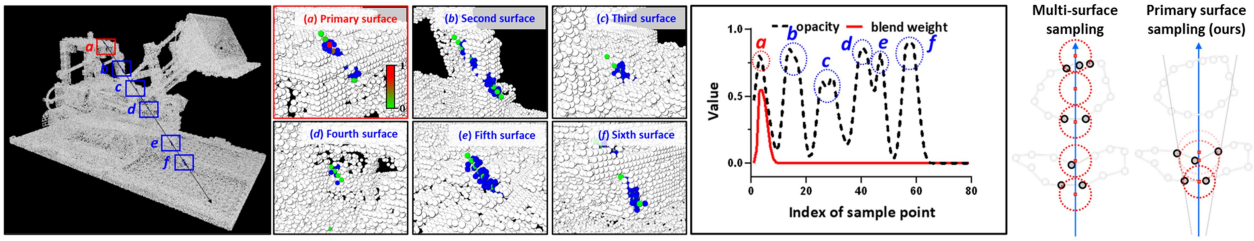
This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

HashPoint: Accelerated Point Searching and Sampling for Neural Rendering

Jiahao Ma[1,2], Miaomiao Liu[1], David Ahmedt-Aristizabal[2], Chuong Nguyen[2]
Australian National University[1], CSIRO Data61[2]
{jiahao.ma, miaomiao.liu}@anu.edu.au
{jiahao.ma, david.ahmedtaristizabal, chuong.nguyen}@data61.csiro.au



(a) HashPoint combines rasterization and ray-tracking approaches to optimize point searching and adaptive sampling on primary surfaces. **(A)** A ray intersects at six surfaces *a*∼*f*. **(B)** Zoomed-in surfaces to show sample points in green and red, with the nearby retrieved point cloud in blue. A shift from green to red represents increasing blend weight of the sample points. **(C)** The graph illustrates opacity and rendering weight changes along a camera ray. The first peak, labeled '*a*', indicates the *primary surface* where the ray first hit. Higher rendering weights mean more significant sample points. Notably, the primary surface often dominates the rendering process. **(D)** Sampling across multiple surfaces versus our HashPoint which accelerates the process via sampling solely on the primary surface.

Abstract

In this paper, we address the problem of efficient point searching and sampling for volume neural rendering. Within this realm, two typical approaches are employed: rasterization and ray tracing. The rasterization-based methods enable real-time rendering at the cost of increased memory and lower fidelity. In contrast, the ray-tracing-based methods yield superior quality but demand longer rendering time. We solve this problem by our HashPoint method combining these two strategies, leveraging rasterization for efficient point searching and sampling, and ray marching for rendering. Our method optimizes point searching by rasterizing points within the camera's view, organizing them in a hash table, and facilitating rapid searches. Notably, we accelerate the rendering process by adaptive sampling on the primary surface encountered by the ray. Our approach yields substantial speed-up for a range of state-of-the-art ray-tracing-based methods, maintaining equivalent or superior accuracy across synthetic and real test datasets. The code will be available at https://jiahao-ma.github.io/hashpoint/.

1. Introduction

Photo-realistic rendering, a significant challenge in computer vision, has seen notable advancements with NeRF [36] and its extensions [4, 5, 33, 65]. Approaches in [14, 36, 36, 61, 65] rely on global MLPs to reconstruct radiance fields across the entire space through ray marching. However, this method results in slow per-scene neural network fitting and extensive, often unnecessary, sampling of vast empty space, leading to prohibitive reconstruction and rendering times. To overcome this challenge, point clouds are introduced as a straightforward representation of surfaces in space, forming point-based neural radiance fields [2, 10, 11, 23, 25, 41, 46, 48, 62, 66]. Point clouds approximate scene geometry and encode appearance features, expediting the rendering process by sampling and aggregating features near multiple surfaces represented by the point cloud. Existing methods in this field typically adopt two rendering strategies: rasterization and ray tracing.

In the rasterization framework [2, 8, 17, 23–25, 32, 46, 64, 66], points are initially projected onto the image plane, where pixel values are determined by the color of the nearest point using z-buffer mechanism. While this approach enables real-time rendering, it often exhibits visible holes stemming from the density of the point cloud. To address this limitation, some methods [2, 13, 23, 46, 66] leverage networks such as U-Net [47] for hole filling by employing feature downsampling and upsampling. However, this approach struggles to generate high-fidelity details. Alternative strategies are to assign each point as a 3D shape such as an oriented disk [17, 24, 32, 42, 64] or a sphere [27]. Nevertheless, determining the optimal size for these shapes is a challenging task. Small shapes can lead to gaps in the rendering, while larger ones may introduce artifacts. Additionally, significant memory is often required for storage.

Ray-tracing-based point cloud rendering [10, 20, 21, 41, 48, 62, 70], directly casts rays onto a point cloud and interpolates nearby points' features along each camera ray, addressing issues like holes found in rasterization-based methods and enabling high-fidelity novel view synthesis. However, real-time rendering poses challenges due to the complex point cloud search process and excessive sampling of multiple surfaces. To enhance point cloud searching efficiency, methods in [10, 41, 48, 62] employ accelerated data structure like Uniform grid [18], K-d tree [16, 19, 68], Octree [34, 49] or bounding volume hierarchies (BVH) [56]. However, not all surface features are equally important for rendering, as illustrated in Figure 1a. Typically, only surfaces closest to the camera significantly contribute to rendering, making subsequent sampling redundant. Other methods attempt to collect the K nearest points for each ray to predict colors [10, 41], struggling with sparse point density for extracting primary surface features.

In this work, we introduce **HashPoint**, an optimized point cloud searching approach designed to address the hole issue and accelerate the search process by adaptive sampling on primary surfaces. The core of our method involves transforming the 3D search space into a 2D image plane for hash table lookup. Unlike traditional rasterization methods using the z-buffer to retain only the nearest points, we project all points onto the image plane and preserve all points within each pixel. These points are then stored in a hash table. This accelerated structure enables a swift point location near the camera ray through the hash table lookup. What distinguishes our approach is the adaptive searching range, determined by the distance between points and the viewpoint, rather than relying on a fixed radius or K nearest points. After identifying the point clouds near the camera ray, these points are projected onto the ray, with each projected point serving as a potential sample point candidate. The selection of sample points is crucial, as it directly influences the feature aggregation quality on the primary surface. We calculate the importance of each sample point candidate based on the distance between the point cloud and the candidate itself. Drawing inspiration from volume rendering techniques, we retain high-importance sample point candidates, those closer to the viewpoint, becoming the definitive sample points. Consequently, the number of sample points for each ray varies adaptively, ranging from 0 to n, ensuring a dynamic sampling process. In summary, our main contributions consist of two key techniques:

- We introduce *Hashed-Point Searching* as a novel technique that accelerates the ray-tracing approach by optimizing point searching for improved efficiency.
- We also propose a novel technique called *Adaptive Primary Surface Sampling* to adaptively sample on the first encountered surface by the ray determined by the distance between points and the viewpoint.
- We validate our approach on various benchmark datasets (synthetic, real, indoor, and outdoor), demonstrating its significant potential to accelerate the rendering process by a large margin with similar or better accuracy.

2. Related work

Point cloud rasterization. Rasterization is a widely used technique for rendering point clouds. The basic concept involves projecting each point onto the image plane while ensuring that closer points occlude those that are farther away via the z-buffer mechanism. A significant challenge in point-cloud rasterization is the unwanted occurrence of gaps or holes in the rendered output. Classical methods such as visibility splatting [42, 71] address these gaps by substituting points with oriented disks. However, determining the optimal size and shape of these disks is complex, and they may not always completely cover the visible gaps in the image. In contrast, recent methods such as differentiable splatting, employed by [17, 24, 35, 64], have made a notable improvement in rendering quality by fine-tuning the shape of these disks through optimization. Recent advancements in the field involve the integration of rasterization with neural networks. NPBG [2] and NPBG++ [46] employ U-Net [47] refinement to learn the rasterizing features, minimizing the disparity between the rendered images and ground truth. These approaches address the issue of holes by leveraging both feature downsampling and upsampling. Huang *et al.* [23] introduce radiance mapping as a means to combat spatial frequency collapse. However, this technique still encounters difficulties in generating high-fidelity details in coarse regions. As a solution, FreqPCR [66] introduces an adaptive frequency modulation module designed to capture the local texture frequency information.

Ray tracing for point cloud. This presents an alternative approach for rendering point clouds. Early works [1, 30, 55] developed iterative strategies to determine ray intersections with surfaces approximated from point clouds. Recently, methods such as NPLF [41] focus on embedding features at each point and aggregating them during a query, while Pointersect [10] directly determines ray intersections with the inherent surface. PAPR [67] employs point cloud positions to capture scene geometry, even when the initial geometry substantially differs from the target geometry. However, this method mainly focuses on the nearest points around the ray, potentially missing the features of the primary surface, particularly when dealing with sparse point clouds. On the other hand, Point-SLAM [48] and Point-NeRF [62] uniformly sample across multiple surfaces to produce high-quality renderings. Despite avoiding unnecessary sampling of vast empty space, such processes of extensive feature aggregation and MLP prediction on subsequent surfaces remain time-consuming. Thus, there is a need for selective sampling on correct surfaces to improve both accuracy and efficiency in point cloud rendering.

Efficiency for neural radiance field. Recent works [28, 38, 59, 60] improve the memory and optimization efficiency via hash encoding. Some other works [26, 39, 43, 52, 57] are designed to improve the sampling efficiency of NeRF while maintaining a compact memory footprint. DONeRF [39] improves sampling efficiency with a depth-trained oracle network. TermiNeRF [43] uses a density-based sampling network from a pre-trained NeRF. AutoInt [52] predicts segment lengths along rays for sampling. AdaNeRF [26] employs a dual-network architecture with incremental sparsity for fewer samples of higher quality. In general, these methods utilize pre-trained modules and complex optimization for sample point distribution along rays. In addition to neural network-assisted sampling, some other approaches [12, 15, 29, 44] adopt multi-view stereo (MVS) technique [51, 63] or depth sensor to obtain depth information[48], subsequently sampling only on the surfaces of scenes, closely resembling ray-tracing based point cloud rendering. Yet, they rely on dense depth map input and tend to encounter inefficiencies from oversampling on multiple surfaces during rendering. This paper presents our pioneer concept of sampling on the primary surface, further optimizing the use of the explicit geometric representation.

3. Method

Following preliminaries in Section 3.1, we detail our Hash-Point method with our point searching technique in Section 3.2, and our surface sampling approach in Section 3.3, and then integrate these with existing methods in Section 3.4. Comparative analysis is explained in Section 3.5.

3.1. Preliminaries of neural rendering

Point cloud rendering based on ray tracing typically employs two strategies to ascertain the color of each ray:

Image-based rendering. Given a point cloud denoted as $\mathcal{P} = \{(x_i^{pc}, c_i^{pc}, f_i)\}_{i=1...n}$, where $x_i^{pc} \in \mathbb{R}^3$ represents position, $c_i^{pc} \in \mathbb{R}^3$ is RGB color and f_i depicts high-dimensional appearance feature. Several methods [10, 41] predict the blending weight $w_i(\mathbf{r}, \mathcal{P})$ for camera ray $\mathbf{r}$ utilizing the appearance feature, while others, like [41, 67], use the geometric relationship with the ray. The final color of a camera ray $c(\mathbf{r})$ can be computed by K nearest points, represented as *K-NP* in Figure 4A:

$$c(\mathbf{r}) = \sum_{i=1}^{K} w_i(\mathbf{r}, \mathcal{P}) c_i^{pc}. \tag{1}$$

Although methods [10, 41, 67] exhibit some variations, fundamentally, they all harness the relationship between rays and proximate points to determine color.

Volume rendering. Unlike predicting colors from neighboring points of a ray, some methods [48, 62] uniformly sample on surfaces, and aggregate point features f_i to sampled points, as shown in Figure 4B. These methods subsequently employ physically-based volume rendering to compute the color of individual pixels. Specifically, the radiance of a pixel is derived by traversing a ray through it, sampling N sample points at $\left\{x_j^{sp} \mid j = 1, \ldots, N\right\}$ along the ray, and accumulating radiance using volume density, as:

$$\begin{aligned} c &= \sum_{j=1}^{N} \tau_j \left(1 - \exp\left(-\sigma_j \Delta_t\right)\right) c_j^{sp}, \\ \tau_j &= \exp(-\sum_{k=1}^{j-1} \sigma_k \Delta_k). \end{aligned} \tag{2}$$

Here, τ represents volume transmittance; σ_j and c_j^{sp} correspond to the density and color for each sample point j at x_j^{sp}. Δ_t is the distance between adjacent sample points.

3.2. Hashed-point searching

Accelerated structure construction. In contrast to traditional approaches that perform point cloud searching in 3D space, we innovate by shifting the search from 3D space to a 2D image plane for hash table lookup. As illustrated in Figure 2, our approach diverges from classic rasterization methods that employ the z-buffer to retain the nearest points for each pixel. Instead, we project all points onto the image plane and preserve all points within each pixel, storing them in a hash table for efficient retrieval. Our Hash-Point approach to reorganized point cloud is inspired by the Morton code [37] (often referred to as Z-order), which provides a linear ordering of multi-dimensional data for efficient retrieval and storage. Specifically, we arrange the points falling on the same pixel in a Z-order manner to ensure they are stored as closely as possible in sequence. Therefore, we adjust the order of points in the point cloud list. The positional adjustment of each point is achieved through atomic operations in CUDA [40], with a time complexity of only $\mathcal{O}(1)$ for high efficiency. Following these adjustments, we use a hash table to store the position of each point in the point list. In the hash table, the key is the index of the pixel, and the value includes the position of the first point stored for the current pixel in the point list, along with the number of points that fall on that pixel.

Searching. Inspired by [4, 6, 22] each pixel emits a cone for point cloud searching and feature aggregation, which is more in line with the principles of imaging. The use of a fixed searching radius may include features that do not belong to the pixel, introducing noise. For a detailed comparison, please refer to our Supplementary. In this work, we propose an adaptive searching radius that is proportional to the distance of the sampling point relative to the ray origin.

Following [22], we represent the pixel as a circle on the image plane, as an approximation to the area of the pixel. As shown in Figure 3A, the radius of the disc can be calculated by $\dot{r} = \sqrt{\Delta x \cdot \Delta y / \pi}$, where Δx and Δ_y are the width and height of the pixel in world coordinates, derived from

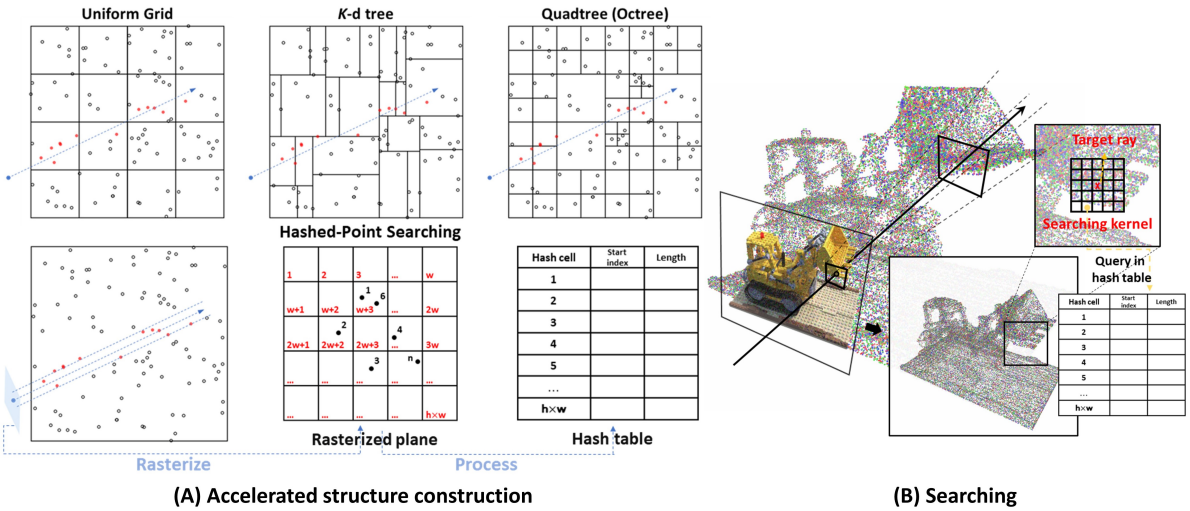


Figure 2. Efficient point clouds searching for ray tracing. **(A) Top row** shows traditional point cloud search strategies: Uniform Grid, *K*-D tree, and Octree visualized in 2D for clarity. **(A) Bottom row** shows *Hashed-Point Searching*, our method of rasterizing the point cloud onto an image plane and then reorganizing the point cloud list to optimize queries, resulting in the construction of a hash table. Each key (as pixel index) in the table maps to the start index in the point list and the count of points within that pixel with $\mathcal{O}(1)$ complexity. **(B)** The final selection depicts the search mechanism: using a magnified search kernel, the neighbor points of a target ray (black arrow) are swiftly identified through the hash table and assessed based on their distance to the ray.

the calibrated camera parameters. For each pixel, a cone emits from the ray origin $\mathbf{o}$ (optical center of the camera) along the ray direction $\mathbf{d}$, passing through the pixel center $\mathbf{p_o}$. Due to the sparsity of the point cloud, there might be no points falling within the ray cone, causing holes in the rendering. To mitigate this, we modify the ray cone for broader coverage, shifting from a disc with radius $\dot{r}$ for one pixel to a larger disc of radius $\ddot{r}$ for the searching kernel. We deduce the magnitude of the searching kernel s by $s = 2 \cdot \lceil \frac{\ddot{r}}{\dot{r}} \rceil + 1$. Any sample point that lies on the ray can be derived by $\mathbf{x_j^{sp}} = \mathbf{o} + t\mathbf{d}$, and the adaptive radius r of sample point can be computed by,

$$r = \frac{\|\mathbf{x_j^{sp}} - \mathbf{o}\|_2 \cdot f\ddot{r}}{\|\mathbf{p_o} - \mathbf{o}\|_2 \cdot \sqrt{\left(\sqrt{\|\mathbf{p_o} - \mathbf{o}\|_2^2 - f^2} - \ddot{r}\right)^2 + f^2}}. \quad (3)$$

Here, f is the focal length. For derivation, please refer to the Supplementary. During the rendering process, we sample between t_{near} and t_{far}. As t increases, the radius of the sample point also increases. As the minimum radius, r_{min}, is at the t_{near} position and matches with our input hyperparameter radius, we modify s accordingly. In the target ray's search kernel, we traverse each pixel and assess the number of associated points per pixel to determine if the search is needed. If required, we look up the point list for further judgment. The query complexity remains at $\mathcal{O}(1)$.

3.3. Adaptive primary surface sampling

Another core of our proposed method lies in the adaptive aggregation of features from the primary surface, guided by the distribution of the nearby point cloud. As shown in

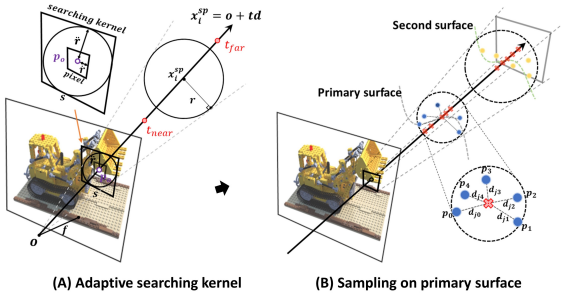


Figure 3. Illustration of *Adaptive Primary Surface Sampling*. **(A)** The diagram depicts the generation of the searching kernel on the image plane. **(B)** We project adjacent points to the ray as sample point candidates (red crosses). Each candidate's importance, determined by its distance distribution to points within its radius, influences its preservation for final feature aggregation.

Figure 3B, neighboring points are projected onto the camera ray, forming a set of sample point candidates denoted as x_j^{sp} where j represents the index of the sample point. The geometric distribution of these sample points is determined by calculating the average distance between each candidate and its K neighboring points. The average distance d_j can be computed by

$$d_j = \frac{1}{K} \sum_{i=1}^{K} \left\| x_j^{sp} - x_i^{pc} \right\|_2. \quad (4)$$

While this distance between the sample point and the surface presented by the point cloud adheres to the principles of the Unsigned Distance Function (UDF) [31], it deviates from a strict UDF definition. Therefore, we term it as

"pseudo-UDF". Drawing inspiration from volume rendering [36] and its extension [31, 58], we adhere to two rules:

1. **Unbiased intersection**. A reduced pseudo-UDF value indicates that the candidate is proximal to the surface, thus deserving a higher weighting.
2. **Occlusion-aware**. When candidates exhibit identical pseudo-UDF values, the one positioned closer to the viewport is assigned a greater weight to the final color output.

To follow the first principle, the distance d_j is transformed into a confidence a_j by $\alpha_j = \gamma \exp\left(-\frac{d_j^2}{\beta^2}\right)$, where β is the hyperparameter which depends on the density of the point cloud and γ is to control the range of sampling. The proximity of the sample point to the point cloud, lesser d_j, necessitates a reduced β to ensure distinct differentiation in the confidence a_j cross sample points. To make the distribution of sample point occlusion aware, based on volume rendering, we define the weight function by

$$w_j = \alpha_j \prod_{k=1}^{j-1} (1 - \alpha_k) \,. \tag{5}$$

We retain the sample point candidates with w_j larger than zero, which are located near the primary surface.

3.4. Integration with existing methods

The proposed method is adaptable to most ray-tracing point cloud rendering techniques. After sampling on the primary surface and identifying surface point cloud, we can use MLP, as in [48, 62], to predict density and color for sample points. The color of the pixel is then rendered by volume rendering. Additionally, we can integrate approaches from [10, 41, 67] to estimate the ray's color based on point features. Please refer to Section 4 for more comparison.

3.5. Comparative analysis

Our hashed-point searching *vs.* traditional point cloud searching. Efficient retrieving neighboring points for a camera ray is non-trivial. A straightforward method is to compare all rays with all points, brute force searching, leading to high computation costs. Common approaches employ space partitioning to speed up the search. Below, taking an example with n points and m rays with an average of q points falling in the radius δ of each ray, we introduce the search processes and the corresponding complexity of these algorithms.

- ***K*-d tree** [7]: a space-partitioning data structure for organizing points in a k-dimensional space. To locate points near a ray using this structure, two common strategies exist: (1) Sample uniformly along the ray followed by radius searches at each sample point, and (2) leverage sub-tree bounding boxes in the *K*-d tree and apply axis-aligned bounding box (AABB [50]) for intersections, then compare points in intersecting sub-trees. Building the *K*-d tree needs to iterate n points with the complexity of $\mathcal{O}(n)$, and in the case of searching points for m rays, the complexity of finding all intersecting sub-trees and retrieving q points within the cylinder of rays is $\mathcal{O}(mlog(n) + mq)$.
- **Octree** [34]: a tree data structure used to partition a three-dimensional space by recursively subdividing it into eight octants. This strategy for finding points near a ray is similar to the *K*-d tree mentioned above, and the complexity is the same - constructing an Octree requires $\mathcal{O}(n)$, while searching requires $\mathcal{O}(mlog(n) + mq)$.
- **Uniform grid** [18]: a data structure that divides space into regular grids and allocates data to each grid cell. Building this data structure requires traversing all points with complexity $\mathcal{O}(n)$. For searches, using algorithms like 3DDA [3] or AABB to find intersecting voxel with rays, the complexity is $\mathcal{O}(mg+mq)$, where g is the number of grid cells per dimension.
- **Hashed-point searching**: In terms of design, prior methods involve the ray actively seeking point cloud in 3D space. In contrast, our method allows the point cloud to seek rays. To accelerate queries, we construct an acceleration structure in the form of a hash table by iterating through all points, followed by searching via hash table lookup. The complexity of this process is $\mathcal{O}(n + mq)$.

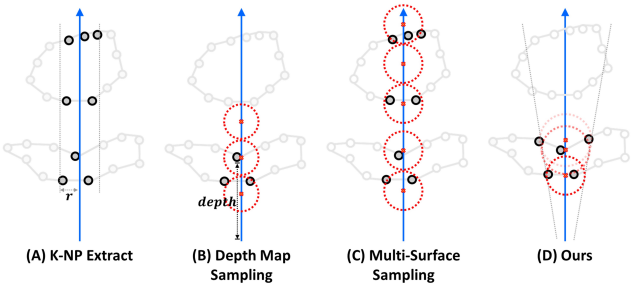


Figure 4. Comparison of different point cloud selection strategies. **(A)** *K-NP Extract*: extracting *K* nearest point features per ray. **(B)** *Depth Map Sampling*: sampling based on dense depth maps. **(C)** *Multi-Surface Sampling*: uniform sampling over multiple surfaces. **(D)** *Our method*: adaptive sampling on the primary surface.

Our adaptive primary surface sampling *vs.* existing point cloud selection strategies. The selection of the number and location of points significantly determines the quality and efficiency of rendering. As shown in Figure 4, we compare various methods of point cloud selection as follows:

- "*K-NP Extract*" [10, 41, 67] refers to extracting the nearest *K* point features around a ray to predict its color. As shown in Figure 6, due to the density of point cloud, the *K* nearest points often don't necessarily fall on the primary surface, leading to feature noise.
- "*Depth Map Sampling*" [44, 48] refers to using sensor

depth or predicted depth via MVS [51, 63] to assist sampling. Although this method is highly efficient, it relies on dense depth maps and also faces the issue of redundant sampling on multiple surfaces (especially in the case of relying on nearby views with depth input).

- "*Multi-Surface Sampling*" [62] involves finding all surfaces intersecting with the ray and then uniformly sampling on them. The method provides the best result at the high cost of computation.
- *"Adaptive Primary Surface Sampling"* gives a good balance between precision and efficiency by sampling on the primary surface, as explained in Sections 3.3, 4.2 and 4.3.

Ours *vs.* 3D Gaussian splatting [24]. While our method shares similarities with 3D Gaussian splatting in utilizing rasterization for point searching, a key distinction lies in our rendering approach. While Gaussians have shape, they are often small, requiring more points for ray rendering. In contrast, our method *interpolates* ray color from nearby points, requiring fewer points. For instance, in experiments with the same Lego scene (NeRF-Synthesis Dataset), our method uses only 35Mb of storage, compared to Gaussian splatting's 200Mb. The proposed method takes advantage of both rasterization and ray tracing, complementing each other. The combination of the two methods further enhances the approach to promising neural rendering.

4. Experiments

4.1. Baselines and integration

This section outlines four key baselines in ray-tracing point cloud rendering, focusing on searching and sampling, as detailed in Table 1. We compare the corresponding components with the proposed enhancement as follows:

- *Point-NeRF* [62]: Searches via uniform grid (*UG*) and then samples across multiple surface (*MS*). We replace the *UG* and *MS* with our HashPoint (*HS*) and primary surface sampling (*PS*) and benchmark as shown in Table 2. *PS* alone impedes the point optimization due to limited gradient propagation. Thus, *MS* is initially used for $10K$ iterations for geometry optimization, then switched to *PS* for boosting efficiency. The transition is controlled by adjusting the parameter γ. For more details, please refer to the supplementary material.
- *Point-SLAM* [48]: Employs single-surface (*SS*) sampling, different from Point-NeRF's *MS* approach. While Point-SLAM's depth-guided sampling is efficient, it relies on dense depth input, not always available during rendering. Our method's performance as shown in Table 3 is compared with both of Point-SLAM's sampling strategies: depth-guided and uniform multi-surface.
- *NPLF* [41]: Starts with downsampling points, searches nearby points using brute force (*BF*), and renders on K nearest points (*K-NP*). Our method maintains high fidelity by selecting K (K = 8) nearest points from the primary surface without downsampling as shown in Table 4.
- *Pointersect* [10]: Searches using a *UG*, retaining K (K = 40) nearest points for rendering. We improve this by selecting K (K = 6) nearest points from primary surfaces, gaining higher speed and quality as shown in Table 5.

Method	Searching	Selection
Point-NeRF [62]	Uniform grid	Multiple surfaces
NPLF [41]	Brute force	K nearest points
Pointersect [10]	Uniform grid	K nearest points
Point-SLAM [48]	Depth map sampling	Single surface

Table 1. Comparison of key components in various baselines.

We evaluate our search and selection modules, along with overall performance, in Sections 4.2 and 4.3. The evaluation metrics include Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), Learned Perceptual Image Patch Similarity (LPIPS), and frames per second (FPS). Our method, integrated with these baselines, is tested on Synthetic-NeRF [36], Waymo [54], Replica [53], and ShapeNet [9] datasets. We adopt their training approach, with a key difference: in Point-NeRF, we refine the geometry, unlike other experiment baselines. This refinement impacts accuracy; for more details, see Section 4.3.

4.2. Evaluation

Evaluation on Synthetic-NeRF [36]. We incorporate the proposed method into Point-NeRF [62] and compare with both NeRF [36] and various point-based approaches [2, 23, 46, 62, 66]. As shown in Table 2, this integration achieves an **80**-fold speedup while ranking as the second-highest performance among point-based methods. The results of Figure 5 qualitatively show that solely sampling on the primary (single) surface can produce multi-surface effects, presenting the right balance between performance and speed for ray-tracing-based methods.

Evaluation on Replica [53]. We also compare the integration of Point-SLAM [48] with voxel-based method [69] and the original [48] on the Replica [53] indoor dataset, solely focusing on rendering rather than tracking and mapping. As outlined in Table 3, our approach outperforms Point-SLAM in both PSNR and SSIM with parity in LPIPS. In terms of speed, we analyzed Point-SLAM's depth-guided and uniform multi-surface sampling (*US*), particularly for depth-unknown scenarios. In contrast to Point-SLAM's fixed $\hat{n}$ point collection ($\hat{n} = 5$), our method dynamically gathers 1 to n points based on nearby point distributions ($n = 4$). The efficiency of our approach is notably superior - 1.8 times faster than depth-guided sampling and 11.5 times faster than *US* sampling.

Evaluation on Waymo [54]. In the evaluation of the Waymo dataset, we substitute the *BF* and *K-NP* with our methods while comparing radiance-based methods [14, 36] and the original [41]. Figure 6 illustrates that our method accurately samples object surfaces, outperforming *K-NP*.

	Radiance-based	Point-based (Rasterization)				Point-based (Ray tracing)	
	NeRF [36]	NPBG [2]	NPBG++ [46]	Huang *et al.* [23]	FreqPCR [66]	Point-NeRF [62]	Point-NeRF + Ours
PSNR ↑	31.01	24.56	28.12	28.96	31.24	**33.31**	33.22
SSIM ↑	0.947	0.923	0.928	0.932	0.950	**0.978**	0.961
LPIPS↓	0.081	0.109	0.076	0.061	0.049	**0.049**	0.055
FPS ↑	0.05	33.64	35.21	**39.67**	39.56	0.12	9.60(×80 speed up)

Table 2. Comparison of our method integrated with Point-NERF with a radiance-based model [36], rasterization-based models [2, 23, 46, 66] and a ray-tracing-based model [62] on the Synthetic-NeRF dataset.

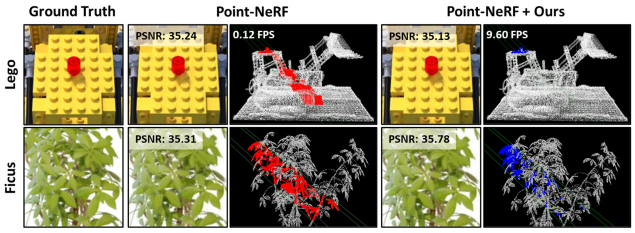


Figure 5. Results on the NeRF-Synthesis [36] dataset shows that our primary surface sampling (blue points) is more efficient than Point-NeRF's sampling (red points) while preserving accuracy.

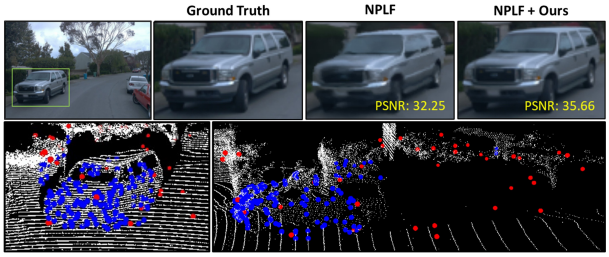


Figure 6. Comparison on Waymo [54] dataset shows how our primary surface sampling (blue points) more accurately samples the car body than the K nearest point selection of NPLF (red points).

Table 4 shows our method's superior accuracy over NPLF and its enhanced efficiency compared to *BF*.

Evaluation on ShapeNet [9]. To evaluate on the ShapeNet dataset without per-scene optimization, we adopt the training setting of Pointersect - training on 48 training meshes from the sketchfab [45] dataset and testing on 30 meshes in ShapeNet. We streamline the architecture of Pointersect by reducing the input of neighbor points (K: *40* to *6*), significantly enhancing the efficiency of inference. The results presented in Table 5 indicate that using only the K nearest points of the primary surface produces robust outcomes and accelerates the process.

Comparison with traditional point cloud search for ray casting. In Figure 7, our method outperforms traditional methods (Uniform grid, K-d tree, and Octree) in point cloud search efficiency for ray casting. All comparisons are conducted on a CPU (*Intel(R) Core(TM) i9-12900K*) without GPU acceleration. The left graph shows our method's superior performance with increasing point numbers, while the right graph demonstrates consistent efficiency with rising ray counts. Notably, all methods retrieve the same number of points, underscoring that differences in performance result from search efficiency. Our proposed method running on a *NVIDIA RTX 4090* GPU for 1 million points only takes 4 ms to complete searching (0.5 ms) and sampling (3.5 ms) proving its high efficiency of processing large-scale point cloud data for ray casting.

	Voxel-based	Point-based (Ray tracing)	
	NICE-SLAM [69]	Point-SLAM [48]	Point-SLAM + Ours
PSNR ↑	24.42	35.17	**35.43**
SSIM ↑	0.809	0.975	**0.983**
LPIPS↓	0.233	**0.124**	0.126
FPS ↑	0.43	0.95($Depth$) | 0.15(US)	**1.72** (×1.8 | ×11.5 speed up)

Table 3. Comparison of our method integrated with Point-SLAM [48] with NICE-SLAM [69] and Point-SLAM on Replica dataset [53]. $Depth$ and US denote depth-guided sampling on a single surface and uniformly sampling across multiple surfaces separately. The speed evaluates the performance of rendering instead of mapping and tracking.

	Radiance-based		Point-based (Ray tracing)	
	NeRF [36]	DS-NeRF[14]	NPLF [41]	NPLF + Ours
PSNR ↑	22.47	26.15	29.96	**30.57**
SSIM ↑	0.700	0.772	0.868	**0.912**
LPIPS↓	0.389	0.310	0.119	**0.105**
FPS ↑	0.11	0.11	0.33	**1.98** (× 6 speed up)

Table 4. Comparison of our method integrated with NPLF [41] with two radiance-based model [14, 36], and a ray-tracing-based model NPLF on the Waymo Open dataset.

	Point-based		
	Rasterization	Ray tracing	
	NPBG++ [46]	Pointersect [10]	Pointersect + Ours
PSNR ↑	19.3 ± 4.0	28.0 ± 6.4	**29.5 ± 5.6**
SSIM ↑	0.8 ± 0.1	1.0 ± 0.0	**1.0 ± 0.0**
LPIPS↓	0.18 ± 0.08	0.04 ± 0.04	**0.02 ± 0.01**
FPS ↑	**33.40**	1.25	10.12 (×8 speed up)

Table 5. Comparison on the ShapeNet [9] dataset shows that the integration of our approach with Pointersect[10] yields improved performance over NPBG++ [46] and Pointersect.

4.3. Ablation Study

Comparison with Point-NeRF [62]. Figure 8A illustrates the comparison between Point-NeRF's (*UG*) and (*MS*), and our (*HS*) and (*PS*). *HS* notably outperforms *UG* by 5 times

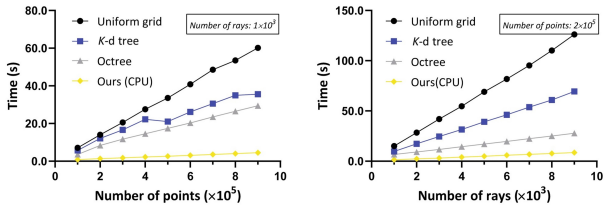


Figure 7. Comparative search performance for neighbor points searching for ray casting: ours *vs.* uniform grid [18], *K*-d tree [7] and Octree [34]. **Left**: search times for various numbers of points with a fixed number of rays. **Right**: search time for a fixed number of points with increasing numbers of rays.

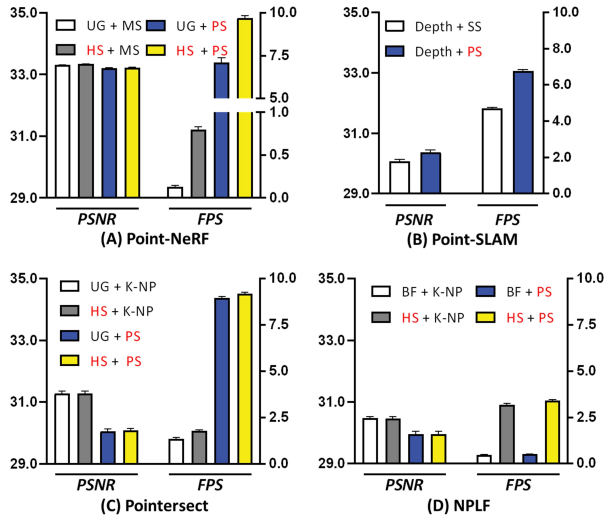


Figure 8. Ablation study NeRF-Synthesis dataset.*HS* and *PS* represent our *Hashed-Point Searching* and *Adaptive Primary Surface Sampling*. **(A)** Combination of *UG* (Uniform Grid) searching with *MS* (Multi-Surfaces) sampling, alongside our techniques. **(B)** Combination of *SS* (single surface sampling) with *Our PS* using *Depth* guided sampling. **(C)** Combination with *UG* and *K-NP* (*K* Nearest Points) selection for sampling. **(D)** Combination with *K-NP* and *BF* (Brute Force) searching.

in speed with *MS*, while *PS* leads to a dramatic speed increase of 60 to 80 times. *HS* and *PS* both increase efficiency without sacrificing accuracy, with *PS* dominating due to reduced point cloud input for feature extraction and MLP prediction.

Comparison with Point-SLAM [48]. Figure 8B presents the advantage of primary surface sampling (*PS*), which adaptively collects 1 to n points on the surface guided by point cloud distribution, over single surface sampling (*SS*), which consistently gather $\hat{n}$ points. The experiment demonstrates that our primary surface sampling not only speeds up the process but also enhances precision.

Comparison with Pointersect [10]. Figure 8C, illustrates that selecting the *K* nearest points (*K-NP*, *K*=40) from noise-initialized point clouds near a camera ray results in higher accuracy compared to using only *PS*. However, our

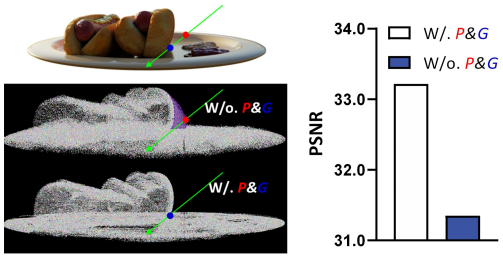


Figure 9. Illustration of geometry refinement necessity. **Left**: Comparison of methods with and without point pruning and growing (*P&G*) for primary surface sampling. Without *P&G*, points (in red) are sampled from a noisy surface (in purple), deviating from the true surface (in blue). **Right**: Performance comparison.

HS method proves faster than the *UG*, contributing significantly to the overall process acceleration.

Comparison with NPLF [41]. Figure 8D shows that in noisy point cloud, the *K* nearest points method (*K-NP*) outperforms primary surface sampling (*PS*). Conversely, Figure 6 illustrates that *PS* is superior to *K-NP* in correct point clouds. Additionally, Hashpoint searching proves to be five times faster than the brute force (*BF*) approach.

Geometry refinement. In the ablation study, we initialized point clouds with Mvsnet[63] and followed Point-NeRF's strategy for optimizing noisy points through point pruning and growing (*P&G*). Geometric optimization was not applied in Point-SLAM, Pointersect, and NPLF, resulting in lower performance. To isolate the algorithm design factors, we test our method with and without refinement on the base of [62]. Figure 9 shows the optimization improves performance, as it mitigates the impact of geometric noise on feature extraction from primary surfaces. Despite relying on geometric structures, it is easily obtained with a brief 10-minute point optimization during training, enabling significant speed gains without losing accuracy. We also explored the combination of γ and β parameters to manage noisy input. Further details are available in our Supplementary.

5. Conclusion

Our method enhances point cloud rendering speed by combining ray tracing with rasterization. Using our HashPoint technique, point clouds are efficiently organized in a hash table through rasterization, accelerating searches. This approach, coupled with primary surface sampling, reduces input points and leverages geometric distribution, significantly speeding up rendering. For instance, searching through a million points takes only 0.5 ms on a standard GPU. In selection, our approach outperforms *K* nearest point methods in accuracy and is faster than multi-surface sampling. Easily integrated with existing methods, HashPoint advances accuracy and rendering speed, pushing the boundaries of point cloud rendering.

References

[1] Bart Adams, Richard Keiser, Mark Pauly, Leonidas J Guibas, Markus Gross, and Philip Dutré. Efficient raytracing of deforming point-sampled surfaces. In *Computer Graphics Forum*, pages 677–684, 2005. 2

[2] Kara-Ali Aliev, Artem Sevastopolsky, Maria Kolos, Dmitry Ulyanov, and Victor Lempitsky. Neural point-based graphics. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXII 16*, pages 696–712. Springer, 2020. 1, 2, 6, 7

[3] John Amanatides, Andrew Woo, et al. A fast voxel traversal algorithm for ray tracing. In *Eurographics*, pages 3–10. Citeseer, 1987. 5

[4] Jonathan T Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5855–5864, 2021. 1, 3

[5] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5470–5479, 2022. 1

[6] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. *arXiv preprint arXiv:2304.06706*, 2023. 3

[7] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975. 5, 8

[8] Ang Cao, Chris Rockwell, and Justin Johnson. Fwd: Real-time novel view synthesis with forward warping and depth. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15713–15724, 2022. 1

[9] Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015. 6, 7

[10] Jen-Hao Rick Chang, Wei-Yu Chen, Anurag Ranjan, Kwang Moo Yi, and Oncel Tuzel. Pointersect: Neural rendering with cloud-ray intersection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8359–8369, 2023. 1, 2, 3, 5, 6, 7, 8

[11] Zhang Chen, Zhong Li, Liangchen Song, Lele Chen, Jingyi Yu, Junsong Yuan, and Yi Xu. Neurbf: A neural fields representation with adaptive radial basis functions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4182–4194, 2023. 1

[12] David Dadon, Ohad Fried, and Yacov Hel-Or. Ddnerf: Depth distribution neural radiance fields. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 755–763, 2023. 3

[13] Peng Dai, Yinda Zhang, Zhuwen Li, Shuaicheng Liu, and Bing Zeng. Neural point cloud rendering via multi-plane projection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7830–7839, 2020. 1

[14] Kangle Deng, Andrew Liu, Jun-Yan Zhu, and Deva Ramanan. Depth-supervised nerf: Fewer views and faster training for free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12882–12891, 2022. 1, 6, 7

[15] Arnab Dey, Yassine Ahmine, and Andrew I Comport. Mip-nerf rgb-d: Depth assisted fast neural radiance fields. *arXiv preprint arXiv:2205.09351*, 2022. 3

[16] Tim Foley and Jeremy Sugerman. Kd-tree acceleration structures for a gpu raytracer. In *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware*, pages 15–22, 2005. 2

[17] Yiming Gao, Yan-Pei Cao, and Ying Shan. Surfelnerf: Neural surfel radiance fields for online photorealistic reconstruction of indoor scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 108–118, 2023. 1, 2

[18] Andrew S Glassner. *An introduction to ray tracing*. Morgan Kaufmann, 1989. 2, 5, 8

[19] Daniel Reiter Horn, Jeremy Sugerman, Mike Houston, and Pat Hanrahan. Interactive kd tree gpu raytracing. In *Proceedings of the 2007 symposium on Interactive 3D graphics and games*, pages 167–174, 2007. 2

[20] Tao Hu, Xiaogang Xu, Ruihang Chu, and Jiaya Jia. Trivol: Point cloud rendering via triple volumes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20732–20741, 2023. 2

[21] Tao Hu, Xiaogang Xu, Shu Liu, and Jiaya Jia. Point2pix: Photo-realistic point cloud rendering via neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8349–8358, 2023. 2

[22] Wenbo Hu, Yuling Wang, Lin Ma, Bangbang Yang, Lin Gao, Xiao Liu, and Yuewen Ma. Tri-miprf: Tri-mip representation for efficient anti-aliasing neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19774–19783, 2023. 3

[23] Xiaoyang Huang, Yi Zhang, Bingbing Ni, Teng Li, Kai Chen, and Wenjun Zhang. Boosting point clouds rendering via radiance mapping. In *Proceedings of the AAAI conference on artificial intelligence*, pages 953–961, 2023. 1, 2, 6, 7

[24] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (ToG)*, 42(4):1–14, 2023. 1, 2, 6

[25] Georgios Kopanas, Julien Philip, Thomas Leimkühler, and George Drettakis. Point-based neural rendering with per-view optimization. In *Computer Graphics Forum*, pages 29–43. Wiley Online Library, 2021. 1

[26] Andreas Kurz, Thomas Neff, Zhaoyang Lv, Michael Zollhöfer, and Markus Steinberger. Adanerf: Adaptive sampling for real-time rendering of neural radiance fields. In *European Conference on Computer Vision*, pages 254–270. Springer, 2022. 3

[27] Christoph Lassner and Michael Zollhofer. Pulsar: Efficient sphere-based neural rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1440–1449, 2021. 1

[28] Yongjae Lee, Li Yang, and Deliang Fan. Mixnerf: Memory efficient nerf with feature mixed-up hash table. *arXiv preprint arXiv:2304.12587*, 2023. 3

[29] Haotong Lin, Sida Peng, Zhen Xu, Yunzhi Yan, Qing Shuai, Hujun Bao, and Xiaowei Zhou. Efficient neural radiance fields for interactive free-viewpoint video. In *SIGGRAPH Asia 2022 Conference Papers*, pages 1–9, 2022. 3

[30] Lars Linsen, Karsten Müller, and Paul Rosenthal. Splat-based ray tracing of point clouds. 2007. 2

[31] Xiaoxiao Long, Cheng Lin, Lingjie Liu, Yuan Liu, Peng Wang, Christian Theobalt, Taku Komura, and Wenping Wang. Neuraludf: Learning unsigned distance fields for multi-view reconstruction of surfaces with arbitrary topologies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20834–20843, 2023. 4, 5

[32] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis. *arXiv preprint arXiv:2308.09713*, 2023. 1

[33] Ricardo Martin-Brualla, Noha Radwan, Mehdi SM Sajjadi, Jonathan T Barron, Alexey Dosovitskiy, and Daniel Duckworth. Nerf in the wild: Neural radiance fields for unconstrained photo collections. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7210–7219, 2021. 1

[34] Donald Meagher. Geometric modeling using octree encoding. *Computer graphics and image processing*, 19(2):129–147, 1982. 2, 5, 8

[35] Marko Mihajlovic, Silvan Weder, Marc Pollefeys, and Martin R Oswald. Deepsurfels: Learning online appearance fusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14524–14535, 2021. 2

[36] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 1, 5, 6, 7

[37] Guy M Morton. A computer oriented geodetic data base and a new technique in file sequencing. 1966. 3

[38] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics (ToG)*, 41(4):1–15, 2022. 3

[39] Thomas Neff, Pascal Stadlbauer, Mathias Parger, Andreas Kurz, Joerg H Mueller, Chakravarty R Alla Chaitanya, Anton Kaplanyan, and Markus Steinberger. Donerf: Towards real-time rendering of compact neural radiance fields using depth oracle networks. In *Computer Graphics Forum*, pages 45–59. Wiley Online Library, 2021. 3

[40] NVIDIA, Péter Vingelmann, and Frank H.P. Fitzek. Cuda, release: 10.2.89, 2020. 3

[41] Julian Ost, Issam Laradji, Alejandro Newell, Yuval Bahat, and Felix Heide. Neural point light fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18419–18429, 2022. 1, 2, 3, 5, 6, 7, 8

[42] Hanspeter Pfister, Matthias Zwicker, Jeroen Van Baar, and Markus Gross. Surfels: Surface elements as rendering primitives. In *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, pages 335–342, 2000. 1, 2

[43] Martin Piala and Ronald Clark. Terminerf: Ray termination prediction for efficient neural rendering. In *2021 International Conference on 3D Vision (3DV)*, pages 1106–1114. IEEE, 2021. 3

[44] Malte Prinzler, Otmar Hilliges, and Justus Thies. Diner: Depth-aware image-based neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12449–12459, 2023. 3, 5

[45] Yue Qian, Junhui Hou, Sam Kwong, and Ying He. Pugeo-net: A geometry-centric network for 3d point cloud upsampling. In *European conference on computer vision*, pages 752–769. Springer, 2020. 7

[46] Ruslan Rakhimov, Andrei-Timotei Ardelean, Victor Lempitsky, and Evgeny Burnaev. Npbg++: Accelerating neural point-based graphics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15969–15979, 2022. 1, 2, 6, 7

[47] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*, pages 234–241. Springer, 2015. 1, 2

[48] Erik Sandström, Yue Li, Luc Van Gool, and Martin R Oswald. Point-slam: Dense neural point cloud-based slam. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18433–18444, 2023. 1, 2, 3, 5, 6, 7, 8

[49] Ruwen Schnabel and Reinhard Klein. Octree-based point-cloud compression. *PBG@ SIGGRAPH*, 3, 2006. 2

[50] Philip Schneider and David H Eberly. *Geometric tools for computer graphics*. Elsevier, 2002. 5

[51] Steven M Seitz, Brian Curless, James Diebel, Daniel Scharstein, and Richard Szeliski. A comparison and evaluation of multi-view stereo reconstruction algorithms. In *2006 IEEE computer society conference on computer vision and pattern recognition (CVPR'06)*, pages 519–528. IEEE, 2006. 3, 6

[52] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. Autoint: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1161–1170, 2019. 3

[53] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, et al. The replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019. 6, 7

[54] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2446–2454, 2020. 6, 7

[55] Ingo Wald and Hans-Peter Seidel. Interactive ray tracing of point-based models. In *ACM SIGGRAPH 2005 Sketches*, pages 54–es. 2005. 2

[56] Ingo Wald, Solomon Boulos, and Peter Shirley. Ray tracing deformable scenes using dynamic bounding volume hierarchies. *ACM Transactions on Graphics (TOG)*, 26(1):6–es, 2007. 2

[57] Huan Wang, Jian Ren, Zeng Huang, Kyle Olszewski, Menglei Chai, Yun Fu, and Sergey Tulyakov. R2l: Distilling neural radiance field to neural light field for efficient novel view synthesis. In *European Conference on Computer Vision*, pages 612–629. Springer, 2022. 3

[58] Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction. *arXiv preprint arXiv:2106.10689*, 2021. 5

[59] Peng Wang, Yuan Liu, Zhaoxi Chen, Lingjie Liu, Ziwei Liu, Taku Komura, Christian Theobalt, and Wenping Wang. F2-nerf: Fast neural radiance field training with free camera trajectories. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4150–4159, 2023. 3

[60] Yiming Wang, Qin Han, Marc Habermann, Kostas Daniilidis, Christian Theobalt, and Lingjie Liu. Neus2: Fast learning of neural implicit surfaces for multi-view reconstruction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3295–3306, 2023. 3

[61] Zirui Wang, Shangzhe Wu, Weidi Xie, Min Chen, and Victor Adrian Prisacariu. Nerf–: Neural radiance fields without known camera parameters. *arXiv preprint arXiv:2102.07064*, 2021. 1

[62] Qiangeng Xu, Zexiang Xu, Julien Philip, Sai Bi, Zhixin Shu, Kalyan Sunkavalli, and Ulrich Neumann. Point-nerf: Point-based neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5438–5448, 2022. 1, 2, 3, 5, 6, 7, 8

[63] Yao Yao, Zixin Luo, Shiwei Li, Tian Fang, and Long Quan. Mvsnet: Depth inference for unstructured multi-view stereo. In *Proceedings of the European conference on computer vision (ECCV)*, pages 767–783, 2018. 3, 6, 8

[64] Wang Yifan, Felice Serena, Shihao Wu, Cengiz Öztireli, and Olga Sorkine-Hornung. Differentiable surface splatting for point-based geometry processing. *ACM Transactions on Graphics (TOG)*, 38(6):1–14, 2019. 1, 2

[65] Kai Zhang, Gernot Riegler, Noah Snavely, and Vladlen Koltun. Nerf++: Analyzing and improving neural radiance fields. *arXiv preprint arXiv:2010.07492*, 2020. 1

[66] Yi Zhang, Xiaoyang Huang, Bingbing Ni, Teng Li, and Wenjun Zhang. Frequency-modulated point cloud rendering with easy editing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 119–129, 2023. 1, 2, 6, 7

[67] Yanshu Zhang, Shichong Peng, Alireza Moazeni, and Ke Li. Papr: Proximity attention point rendering. *arXiv preprint arXiv:2307.11086*, 2023. 2, 3, 5

[68] Kun Zhou, Qiming Hou, Rui Wang, and Baining Guo. Real-time kd-tree construction on graphics hardware. *ACM Transactions on Graphics (TOG)*, 27(5):1–11, 2008. 2

[69] Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R Oswald, and Marc Pollefeys. niceslam: Neural implicit scalable encoding for slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12786–12796, 2022. 6, 7

[70] Yiming Zuo and Jia Deng. View synthesis with sculpted neural points. *arXiv preprint arXiv:2205.05869*, 2022. 2

[71] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Surface splatting. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, pages 371–378, 2001. 2